

Módulo 6. Caso end-to-end y Governance

Curso MLOps/DataOps · ANBAN

José Picón

2026

1. Más allá del MVP: gobernanza del modelo

Hasta aquí has visto cómo construir un sistema MLOps funcional. Falta un capítulo que casi siempre se olvida: la **gobernanza**. Es lo que diferencia un proyecto que vive seis meses de uno que vive seis años.

La gobernanza cubre cinco bloques:

1. Documentación del modelo (Model Card).
2. Trazabilidad y auditoría.
3. Cumplimiento legal (RGPD, AI Act).
4. Sesgo, equidad y subgrupos.
5. Reproducibilidad estricta.

2. La Model Card

Una **Model Card** es un documento estandarizado que describe un modelo. Lo propusieron Mitchell et al. en Google en 2019 y se ha convertido en un estándar de facto.

2.1 Qué debe contener

Sección	Contenido
Propósito	Para qué se entrena y se sirve el modelo
Datos	Origen, periodo, tamaño, sesgos conocidos del dataset
Métricas globales	Accuracy, F1, etc., en test
Métricas por subgrupo	F1 por sexo, edad, raza, geografía
Limitaciones	Casos en los que el modelo no funciona bien
Contraindicaciones	Casos en los que NO se debe usar
Mantenedor	Quién es responsable
Versionado	Hash de commit, versión, fecha

Sin Model Card, dentro de un año nadie sabrá qué hacía ese modelo.

2.2 Generarla automáticamente

Lo que **no se debe hacer**: redactar la Model Card a mano cada vez. Se olvida, se desactualiza, queda obsoleta.

Lo que **sí se debe hacer**: generar las partes automatizables a partir del Model Registry (métricas, run_id, commit) y rellenar a mano solo las partes que requieren criterio humano (limitaciones, contraindicaciones, decisiones de gobernanza).

En el Lab 5 verás un script que hace exactamente esto.

3. Audit trail

Audit trail es el registro completo de qué predicciones hizo cada modelo, cuándo, con qué inputs. En sectores regulados (banca, salud, seguros, justicia) es obligatorio. En cualquier sector es útil para depurar.

Mínimo viable de un audit trail:

- Cada petición a `/predict` se loguea con:
 - `request_id` único.
 - Timestamp ISO 8601.
 - Versión del modelo que la atendió.
 - Inputs (puede ser resumido o hasheado si hay PII).
 - Output (label + proba).
- Los logs se mandan a un sistema centralizado (ELK, Splunk, CloudWatch Logs, etc.).
- Se conservan según regulación local (en banca europea, mínimo cinco años).

En la API del Lab 3 ya guardamos `request_id` y la respuesta lleva `model_uri` y `latency_ms`. Falta enviarlo a un log centralizado.

4. RGPD y AI Act

4.1 RGPD (Reglamento General de Protección de Datos)

Aplica en toda la Unión Europea desde 2018. El artículo más relevante para ML es el **Artículo 22**:

“El interesado tendrá derecho a no ser objeto de una decisión basada únicamente en el tratamiento automatizado, incluida la elaboración de perfiles, que produzca efectos jurídicos en él o le afecte significativamente.”

Si tu modelo toma decisiones automatizadas que afectan a personas (denegar crédito, contratar, asignar becas), el afectado tiene derecho a:

- 1. **Intervención humana** en la decisión.
- 2. **Expresar su punto de vista**.
- 3. **Impugnar la decisión**.

4.2 AI Act europeo

Entró en vigor escalonado en 2024-2026. Clasifica los sistemas de IA por nivel de riesgo:

Nivel de riesgo	Qué incluye	Obligaciones
Inaceptable	Sistemas prohibidos (puntuación social tipo China)	No se permiten
Alto	Scoring crediticio, RRHH, justicia, dispositivos médicos	Documentación, transparencia, registro, supervisión humana
Limitado	Chatbots, deepfakes	Etiquetado obligatorio
Mínimo	Filtros antispam, recomendadores triviales	Sin obligaciones específicas

Si tu modelo es de “alto riesgo”, la Model Card no es opcional: forma parte de la documentación obligatoria.

5. Sesgo y fairness

Una métrica global esconde disparidades. Calcula siempre:

- F1, precision y recall **por subgrupo** (sex, race, edad).
- Tasa de falsos positivos por subgrupo.
- Disparate impact (proporción de decisiones positivas entre grupos).

Si encuentras disparidad grande, tienes cuatro decisiones posibles:

1. **Reentrenar con técnicas de fairness** (reweighing, adversarial debiasing, post-processing).
2. **Cambiar el umbral por subgrupo.**
3. **Documentar la disparidad y aceptarla** (con justificación transparente).
4. **No desplegar el modelo.**

5.1 Decisión humana

Para casos críticos, **siempre** debe haber un humano en el bucle:

- Denegación de crédito.
- Diagnóstico médico.
- Selección de personal.
- Reconocimiento facial en espacios públicos.

El modelo recomienda, un humano decide.

6. Reproducibilidad estricta

Para reconstruir un modelo dentro de un año (por ejemplo, porque un regulador lo pide), necesitas conservar **cinco cosas**:

Componente	Cómo se versiona
Código	git commit hash
Datos	DVC hash MD5
Parámetros	params.yaml o MLflow params
Entorno	requirements.txt + Python version + Docker image hash
Seed aleatoria	random_state fijado en cada librería estocástica

Si tienes los cinco, eres reproducible. Si te falta uno solo, no.

7. Costes y FinOps en ML

Algunos órdenes de magnitud aproximados (varían según cloud):

- Entrenamiento BERT base en GPU cloud: 50 a 200 EUR.
- Inferencia en CPU vs GPU: hasta 100x diferencia de coste para modelos pequeños.
- Endpoint dedicado 24/7 vs serverless: hasta 10x diferencia para cargas bajas.

Preguntas que tu equipo debería responder antes de elegir arquitectura:

1. ¿Online o batch? Si toleras 1 hora de latencia, batch es 10x más barato.
2. ¿GPU o CPU? Para modelos pequeños o medianos, CPU sobra.
3. ¿Autoscaling? Si tu tráfico es 9-18h, no pagues 24/7.
4. ¿Spot instances? Hasta 70 % de descuento para cargas tolerantes a interrupción.

8. LLMOps en un vistazo

Si trabajas con modelos de lenguaje, aplica todo lo anterior más unos cuantos términos nuevos:

Término	Significado
Prompt versioning	Cada prompt es código y se versiona como tal
Eval	Medir la salida de un LLM es difícil. Herramientas: Promptfoo, LangSmith, Ragas
Guardrails	Validar inputs y outputs (Guardrails AI, NeMo Guardrails)
RAG (Retrieval-Augmented Generation)	Conectar el LLM a tu base documental
Vector DB	Pinecone, Weaviate, pgvector, Qdrant

No los tocamos en este curso. Si te interesa profundizar, hay un curso aparte específico de LLMOps.

9. Antes de la práctica: requisitos

Para hacer el ejercicio final del curso necesitas:

1. **Los labs 1, 2, 3 y 4 completados.**
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab5_e2e/`.**

10. Ejercicio final: recorrido end-to-end + Model Card

Objetivo del ejercicio: verificar que tu pipeline MLOps funciona de principio a fin, generar la Model Card del modelo en Staging y rellenar a mano las secciones que requieren juicio humano.

10.1 Paso 1 — Comprobar que todas las piezas están vivas

Antes de ejecutar nada, comprueba el estado del laboratorio:

```
./doctor.sh
```

Tiene que terminar con “Todo en orden”. Si no, arregla lo que diga antes de seguir.

10.2 Paso 2 — Verificar la cadena de artefactos

Comprueba que cada pieza de los labs anteriores está en su sitio:

Lab 1 — Datos:

```
ls -lh ../lab1_dataops/data/processed/
```

Debes ver `train.parquet` y `test.parquet`.

Lab 2 — Modelo en Registry:

```
curl -s 'http://localhost:5050/api/2.0/mlflow/registered-models/get?name=heart-failure-clf' | head -20
```

Debes ver el modelo `heart-failure-clf` con al menos una versión en stage `Staging` o `Production`.

Lab 3 — API (opcional, si quieres tenerla viva):

```
curl -s http://localhost:8000/health
```

Si la API no está arriba, no pasa nada para este lab.

Lab 4 — Reporte de drift:

```
ls -lh ../lab4_cicd_monitoring/reports/
```

Debes ver `drift.html` y `drift.json`.

10.3 Paso 3 — Generar la Model Card automáticamente

El script `run.sh` del Lab 5 genera la Model Card a partir del Registry. Las secciones que se generan solas:

- Propósito (texto fijo).
- Datos (texto fijo).
- Algoritmo y framework (extraído de los tags del run).
- Hiperparámetros (extraídos de los params).
- Métricas globales (extraídas de las metrics).
- Run ID y commit de git (extraídos de los tags).

Lo que tendrás que rellenar tú a mano:

- Métricas por subgrupo (F1 por sex, race).
- Decisiones de gobernanza.
- Limitaciones y contraindicaciones específicas a tu caso.

Ejecuta el script asistido:

```
./run.sh
```

Al terminar, mira el fichero generado:

```
cat MODEL_CARD.md
```

10.4 Paso 4 — Calcular métricas por subgrupo (ejercicio práctico)

Esto va a tu cuenta. Carga el modelo y el test set, calcula F1 por sexo. Crea un fichero `subgroup_metrics.py` con este contenido:

```

import os
import mlflow.pyfunc
import pandas as pd
from sklearn.metrics import f1_score

# Configurar MLflow
os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5050"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

# Cargar el modelo desde el Registry
model = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")

# Cargar el test set
test = pd.read_parquet("../lab1_dataops/data/processed/test.parquet")
y_true = test["DEATH_EVENT"]
X = test.drop(columns=["DEATH_EVENT"])

# Predicciones globales
y_pred = (model.predict(X) >= 0.5).astype(int)
print(f"F1 global: {f1_score(y_true, y_pred):.4f}")

# F1 por sexo (en este dataset, 0=mujer, 1=hombre)
for sex_value, label in [(1, "Hombre"), (0, "Mujer")]:
    mask = (X["sex"] == sex_value)
    if mask.sum() > 0:
        f1 = f1_score(y_true[mask], y_pred[mask])
        print(f"F1 sex={label} (n={mask.sum()}) : {f1:.4f}")

# F1 por edad (mayores/menores de 65)
for label, mask in [(">=65 años", X["age"] >= 65), ("<65 años", X["age"] < 65)]:
    if mask.sum() > 0:
        f1 = f1_score(y_true[mask], y_pred[mask])
        print(f"F1 {label} (n={mask.sum()}) : {f1:.4f}")

```

Ejecútalo:

```
python subgroup_metrics.py
```

Observa la diferencia. En este dataset, típicamente:

```

F1 global: 0.5455
F1 sex=Hombre (n=39): 0.5217
F1 sex=Mujer (n=21): 0.5714
F1 ≥65 años (n=33): 0.6087
F1 <65 años (n=27): 0.4286

```

Aquí vemos algo interesante: el modelo predice peor para los pacientes **jóvenes**. Es esperable porque los eventos de muerte por insuficiencia cardíaca son menos frecuentes en menores de 65, así que el modelo tiene menos casos positivos en los que aprender.

Pregúntate: si tu modelo real tuviera esta disparidad, ¿lo desplegarías? ¿Qué harías para mitigarla? Anótalo en la Model Card, sección “Limitaciones”.

10.5 Paso 5 — Rellenar las secciones manuales

Abre `MODEL_CARD.md` en tu editor. Rellena:

- **Contraindicaciones:** marca con `[x]` los usos en los que NO se debe emplear este modelo (los cuatro que vienen por defecto son buenos: scoring crediticio real, selección de personal, decisiones automatizadas RGPD Art. 22, producción real con tráfico).
- **Métricas por subgrupo:** añade las F1 que acabas de calcular.
- **Governance:** marca las casillas que apliquen y añade notas.

Guarda. La Model Card está completa.

10.6 Paso 6 — Checklist final del curso

Asegúrate de marcar lo que has completado:

DataOps

- ☐ Repositorio Git inicializado en `lab1_dataops`.
- ☐ Dataset versionado por DVC, no por Git.
- ☐ Pipeline reproducible (`dvc.yaml`).
- ☐ `train.parquet` y `test.parquet` generados.

Tracking y Registry

- ☐ Experimento con 3 runs en MLflow.
- ☐ Modelo `heart-failure-clf` registrado.
- ☐ Versión 1 en stage Staging.

Serving

- ☐ Imagen Docker construida.
- ☐ API respondiendo a `/health`, `/version`, `/predict`, `/metrics`.
- ☐ Pydantic rechaza entradas inválidas con 422.

CI/CD y drift

- ☐ `reports/drift.html` con resultados visibles.
- ☐ `promote_if_better.py` ejecutado al menos una vez.

Governance

- ☐ `MODEL_CARD.md` generada y completada a mano.
- ☐ F1 por subgrupos calculado.

Si tienes todo marcado, **has completado el curso.**

11. Hoja de ruta de aprendizaje

Nivel	Skills
Junior MLOps	Docker, Git, Python, MLflow básico, FastAPI, CI básico
Mid	Kubernetes, Terraform, una feature store, Prometheus + Grafana + alerting, un orquestador (Airflow o Prefect)
Senior	Diseño de plataforma ML completa, multi-cloud, optimización de costes, governance + cumplimiento, multitenancy, edge ML, LLMOps

Pasar de junior a mid suele llevar entre uno y dos años de práctica seria. De mid a senior, otros dos o tres años.

12. Bibliografía y recursos

Libros

- *Designing Machine Learning Systems*, Chip Huyen (O'Reilly, 2022). El más completo y actual.
- *Practical MLOps*, Noah Gift (O'Reilly, 2021). Muy hands-on.
- *Machine Learning Engineering*, Andriy Burkov. Conciso y excelente.
- *Reliable Machine Learning*, Cathy Chen et al. (O'Reilly). SRE aplicado a ML.

Webs

- **ml-ops.org**: hoja de ruta de la comunidad.
- **MLOps Community**: Slack muy activo y blog.
- **Made With ML** (<https://madewithml.com>): tutoriales gratuitos.

Whitepapers fundacionales

- Sculley et al., *Hidden Technical Debt in ML Systems* (2015).
- Google, *MLOps: Continuous delivery and automation pipelines in ML*.
- Mitchell et al., *Model Cards for Model Reporting* (2019).